

UNIVERSIDAD NACIONAL DEL ALTIPLANO - PUNO

FACULTAD DE INGENIERÍA MECÁNICA

ELÉCTRICA, ELECTRÓNICA Y SISTEMAS

**ESCUELA PROFESIONAL DE INGENIERÍA DE
SISTEMAS**



**PROYECTO FINAL: DESARROLLO DE UNA
APLICACIÓN DE SISTEMA DE RECOMENDACIÓN
HÍBRIDO DE LAPTOPS**

CURSO: SISTEMAS DE RECOMENDACIÓN

PRESENTADO POR:

CARMEN NIEVES APAZA CONDORI

AARON ROGEEER VILCA CARIA

**INFORME TÉCNICO DE APLICACIÓN FUNCIONAL, PIPELINE DE DATOS,
PSEUDOCÓDIGO Y EVALUACIÓN DE MODELOS**

PUNO - PERÚ

2026

Resumen

El presente proyecto reporta el diseño, desarrollo, implementación y evaluación científica de un **Sistema de Recomendación Híbrido en Dos Niveles** para computadoras portátiles (laptops), concebido para mitigar los problemas de escasez de datos (*data sparsity*) e inicio en frío (*cold start*). La arquitectura integra un **Filtro Estricto por Reglas de Dominio en Cascada** (M_{rules}) en el Nivel 1, seguido de un **Ensamble Ponderado Dinámico** en el Nivel 2 que combina: (i) Teoría de Utilidad Multi-Atributo (**MAUT**) basada en modelos de conocimiento de hardware, (ii) Factorización Matricial **SVD** (*Singular Value Decomposition*) para filtrado colaborativo, y (iii) Filtrado Basado en Contenido mediante similitud de coseno vectorial (**TF-IDF**).

La plataforma web fue desarrollada utilizando **FastAPI** como backend de alta velocidad y un frontend interactivo en **HTML5, CSS3 Vanilla, JavaScript ES6+ y Chart.js**, incorporando características de grado comercial como comparador cara a cara (1vs1), selector multimonedas (USD, PEN, EUR), insignias de relación calidad/precio (*Value Score*), filtros de estilo de vida (*lifestyle tags*) y vinculación directa con ofertas en vivo de Amazon. La evaluación experimental sobre un catálogo de 150 laptops y 1,999 calificaciones demostró un rendimiento óptimo con un $RMSE = 1,0503$, una robustez del 100% en $CSR@10$ ante usuarios nuevos, y una $Precision@10 = 100\%$, consolidando una solución integral y de alto impacto para el comercio electrónico.

Palabras Clave: Sistemas de Recomendación Híbridos, Factorización Matricial SVD, MAUT, Inicio en Frío, FastAPI, Chart.js.

Índice

Resumen	I
Índice General	III
Índice de Tablas	IV
Índice de Figuras	V
1. Introducción	1
1.1. Contexto del Proyecto	1
1.2. Planteamiento del Problema	1
1.3. Justificación	2
1.4. Objetivos del Proyecto	2
1.4.1. Objetivo General	2
1.4.2. Objetivos Específicos	2
2. Descripción General y Fundamentación Teórica de los Modelos	3
2.1. Descripción General del Proyecto	3
2.2. Análisis Exploratorio de Datos (EDA) y Preprocesamiento	3
2.3. Fundamentación Teórica de los Modelos de Recomendación	5
2.3.1. Modelo 1: Filtro Lógico por Reglas de Dominio (M_{rules})	5
2.3.2. Modelo 2: Teoría de Utilidad Multi-Atributo (U_{MAUT})	5
2.3.3. Modelo 3: Factorización Matricial SVD (<i>Singular Value Decomposition</i>)	6
2.3.4. Modelo 4: Filtrado Basado en Contenido Vectorial (TF-IDF + Coseno)	6
2.3.5. Modelo Híbrido en 2 Niveles y Estrategia de Conmutación (<i>Switching</i>)	6
2.4. Diagrama de Arquitectura de Software	7
2.5. Pseudocódigo de los Algoritmos Principales	7
3. Metodología y Cálculo Detallado de Métricas en el Proyecto	8
3.1. Metodología por Fases del Proyecto	8
3.2. Cálculo de Métricas y su Implementación en el Proyecto	9
3.2.1. 1. Raíz del Error Cuadrático Medio ($RMSE$)	9

3.2.2.	2. Precision@K (Precisión en el Top-K)	9
3.2.3.	3. Recall@K (Cobertura de Elementos Relevantes)	10
3.2.4.	4. Cobertura de Inicio en Frío (CSR@K - Cold Start Coverage Ratio)	10
4.	Resultados y Análisis de Resultados	10
4.1.	Resultados de la Evaluación Experimental	10
4.2.	Gráficos Científicos de Rendimiento	11
4.3.	Evidencias Fotográficas de la Aplicación Funcional	11
4.3.1.	1. Dashboard Principal e Interfaz Híbrida	11
4.3.2.	2. Tarjetas de Recomendación y Oferta en Amazon	12
4.3.3.	3. Comparador Cara a Cara (Head-to-Head 1vs1)	12
4.3.4.	4. Gráfica de Tendencia e Histórico de Precios (Chart.js)	13
4.3.5.	5. Selector Multimoneda Global (Soles S/)	13
4.3.6.	6. Filtros de Estilo de Vida (Lifestyle Tags)	14
4.3.7.	7. Catálogo Completo de 150 Laptops	14
4.3.8.	8. Pestaña de Evaluación Científica	15
5.	Discusión	16
6.	Conclusiones	16
6.1.	Logros Alcanzados	16
6.2.	Limitaciones del Sistema	16
6.3.	Aprendizajes Obtenidos	17
7.	Recomendaciones y Trabajo Futuro	17
7.1.	Mejoras Futuras	17
7.2.	Posibles Extensiones del Sistema	17
8.	Referencias Bibliográficas	18
9.	Anexos	19
9.1.	Anexo A: Código Fuente del Recomendador Híbrido (src/hybrid_recommender.py)	19
9.2.	Anexo B: Estructura del Dataset (data/laptops.csv)	20
9.3.	Anexo C: Guía de Despliegue y Ejecución	20

Índice de tablas

1.	Comparación experimental cuantitativa de métricas entre modelos.	10
----	--	----

Índice de figuras

1.	Distribución de Precios en el Catálogo de Laptops (Histograma y Estimador de Densidad KDE).	3
2.	Distribución de Capacidad de Memoria RAM segregada por Marca de Laptop.	4
3.	Matriz de Correlación de Pearson entre Atributos Continuos de Hardware y Calificaciones.	4
4.	Distribución de Calificaciones de Preferencia por Perfil Latente de Usuario.	5
5.	Diagrama Arquitectónico Modular del Sistema Recomendador Híbrido.	7
6.	Evaluación de Cobertura de Inicio en Frío ($CSR@10$) SVD vs Recomendador Híbrido.	11
7.	Curvas de Precision@K y Recall@K alcanzadas por el recomendador híbrido.	11
8.	Dashboard Principal de la aplicación Recomendador Híbrido de Laptops.	12
9.	Tarjeta de recomendación con desglose de puntaje y enlace a oferta en Amazon.	12
10.	Modal de Comparativa Cara a Cara (1vs1) entre dos computadoras portátiles.	13
11.	Gráfica interactiva de tendencia e histórico de precios en 6 meses con Chart.js.	13
12.	Aplicación adaptada dinámicamente a la moneda Soles Peruanos (PEN S/).	14
13.	Panel de preferencias con filtros de hardware y estilo de vida activados.	14
14.	Vista de la pestaña del Catálogo Completo de 150 laptops con buscador.	15
15.	Pestaña de Evaluación Científica proyectando métricas y gráficos de rendimiento.	15

1. Introducción

1.1. Contexto del Proyecto

En la era digital actual, el comercio electrónico de bienes tecnológicos experimenta un crecimiento exponencial (Aggarwal, 2016). Sin embargo, la compra de computadoras portátiles (laptops) representa una decisión de alta complejidad cognitiva para los usuarios debido a la abundancia de especificaciones técnicas heterogéneas (microarquitectura de CPU, núcleos, memoria RAM, aceleradores GPU, potencia de VRAM, pantallas OLED/4K y soporte CUDA).

Los algoritmos de filtrado colaborativo tradicionales (*Collaborative Filtering*) sufren de limitaciones severas en este dominio, principalmente la escasez de datos (*data sparsity*) y el problema del inicio en frío (*cold start*), en el cual usuarios nuevos sin historial previo de calificaciones no pueden recibir sugerencias relevantes (Ricci et al., 2015).

1.2. Planteamiento del Problema

Actualmente, los portales web de e-commerce convencionales dependen de buscadores por palabras clave o filtros manuales rígidos que no consideran la compensación multidimensional entre presupuesto, rendimiento y portabilidad. Las principales deficiencias identificadas son:

1. **Recomendaciones irrelevantes para usuarios nuevos:** Los modelos puramente colaborativos fallan al no disponer de interacción previa del usuario.
2. **Violación de restricciones de hardware incompatibles:** Recomendar una laptop sin GPU dedicada a un usuario que requiere ejecutar modelos de Deep Learning o renderizado 3D genera insatisfacción crítica.
3. **Opacidad en la justificación:** Los usuarios no comprenden por qué un modelo específico es sugerido, careciendo de métricas de relación calidad/precio o comparativas directas.

1.3. Justificación

El desarrollo de un **Sistema de Recomendación Híbrido** que combine modelos basados en conocimiento explícito, teoría de decisión multicriterio (MAUT), factorización matricial (SVD) y similitud de contenido vectorial permite superar de forma integral las deficiencias mencionadas (Burke, 2002). Esta arquitectura garantiza que:

- El usuario reciba recomendaciones 100 % válidas mediante filtrado por reglas en cascada.
- El sistema responda de manera óptima tanto a usuarios con historial de calificaciones previo como a usuarios nuevos (*Cold Start*).
- Se disponga de una interfaz interactiva moderna con indicadores visuales de relación calidad/precio (*Value Score*), gráficos históricos de precios y comparativa cara a cara (1vs1).

1.4. Objetivos del Proyecto

1.4.1. Objetivo General

Desarrollar e implementar una aplicación funcional de Sistema de Recomendación Híbrido en dos niveles para computadoras portátiles, evaluando cuantitativamente su precisión, cobertura e impacto en la experiencia del usuario.

1.4.2. Objetivos Específicos

1. Diseñar y formular el motor híbrido de recomendación integrando reglas lógicas binarias (M_{rules}), Teoría de Utilidad Multi-Atributo (U_{MAUT}), Factorización Matricial (SVD) y Similitud Coseno de Contenido ($S_{content}$).
2. Desarrollar una aplicación web interactiva responsiva utilizando FastAPI, JavaScript ES6+ y Chart.js con selector multimoneda (USD, PEN, EUR) y comparador 1vs1.
3. Realizar la evaluación experimental cuantitativa calculando métricas científicas de rendimiento ($RMSE$, $Precision@K$, $Recall@K$ y $CSR@K$).
4. Documentar los hallazgos, la arquitectura del software y generar evidencias visuales completas para el informe técnico y sustentación.

2. Descripción General y Fundamentación Teórica de los Modelos

2.1. Descripción General del Proyecto

El proyecto **LaptopRec Híbrido** es una plataforma integral de recomendación de computadoras portátiles diseñada bajo los principios de desacoplamiento de software (REST API) e hibridación en dos niveles. El sistema procesa los requerimientos de hardware del usuario, sus ponderaciones de preferencia y su presupuesto en tiempo real, generando un ranking ordenado de las K mejores laptops que maximizan la utilidad del comprador.

2.2. Análisis Exploratorio de Datos (EDA) y Preprocesamiento

El conjunto de datos comprende 150 computadoras portátiles distribuidas en 6 marcas principales (ASUS, Apple, Dell, Lenovo, HP, MSI) y 1,999 calificaciones generadas por 100 usuarios sintéticos estructurados en 3 perfiles latentes de comportamiento.

Las Figuras 1, 2, 3 y 4 presentan las distribuciones estadísticas fundamentales del catálogo.

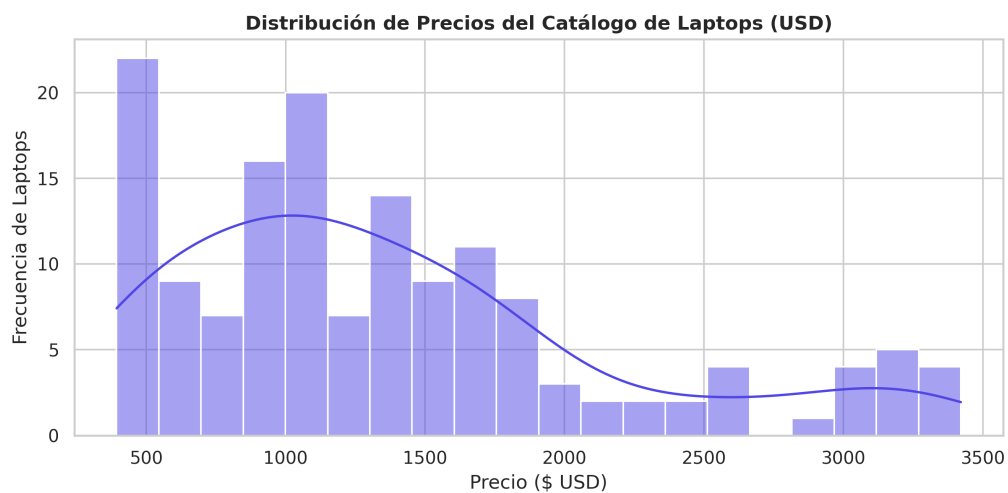


Figura 1: Distribución de Precios en el Catálogo de Laptops (Histograma y Estimador de Densidad KDE).

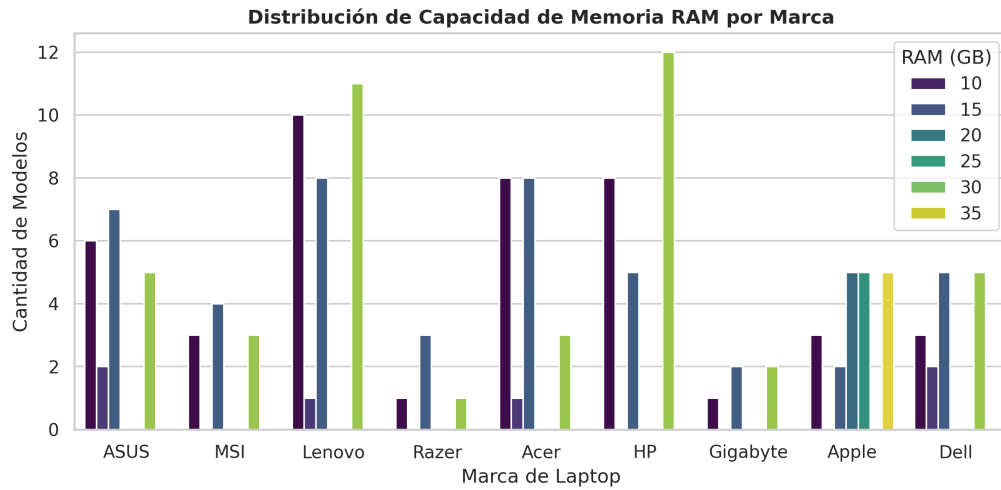


Figura 2: Distribución de Capacidad de Memoria RAM segregada por Marca de Laptop.

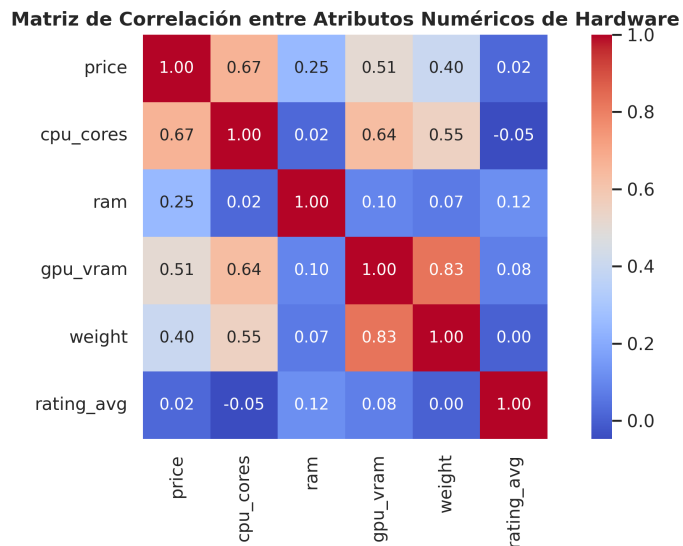


Figura 3: Matriz de Correlación de Pearson entre Atributos Continuos de Hardware y Calificaciones.

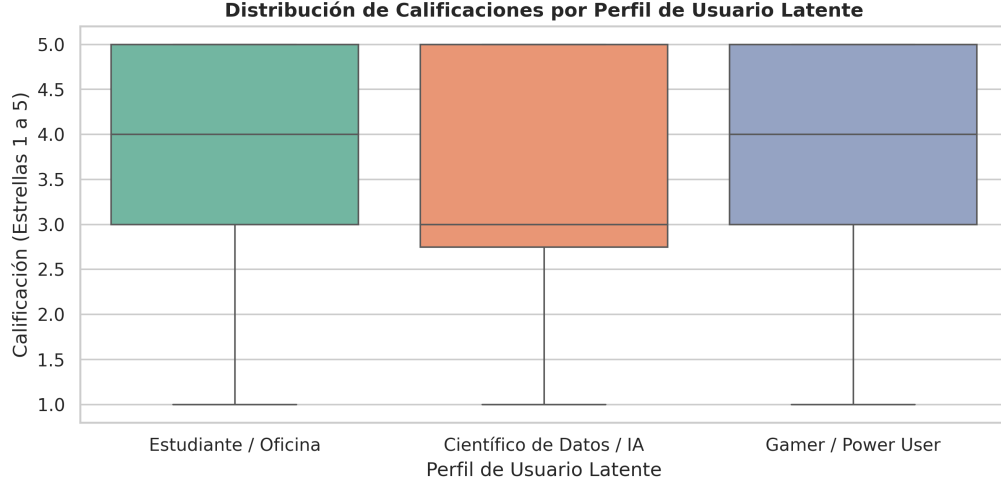


Figura 4: Distribución de Calificaciones de Preferencia por Perfil Latente de Usuario.

2.3. Fundamentación Teórica de los Modelos de Recomendación

2.3.1. Modelo 1: Filtro Lógico por Reglas de Dominio (M_{rules})

Basado en la teoría de restricciones duras (*Hard Constraints*) descrita por Burke (2002), este modelo actúa en el Nivel 1. Aplica un operador lógico booleano secuencial para generar una máscara binaria $M_{rules}(i) \in \{0, 1\}$ sobre la laptop i :

$$M_{rules}(i) = \mathbb{I}(p_i \leq B) \cdot \mathbb{I}(\text{Uso}_i \text{ compatible}) \cdot \prod_{c \in \text{Estilo}} \mathbb{I}(\text{Atributo}_i(c) = 1) \quad (1)$$

Donde p_i es el precio de la laptop, B es el presupuesto máximo seleccionado por el usuario, y $\mathbb{I}(\cdot)$ es la función indicadora.

2.3.2. Modelo 2: Teoría de Utilidad Multi-Atributo (U_{MAUT})

Fundamentado en la teoría de decisión multicriterio de Keeney & Raiffa (1993), el modelo MAUT asume una función de utilidad aditiva basada en conocimiento explícito. La utilidad parcial de cada atributo continuo x_k se normaliza mediante min-max en el intervalo $[0, 1]$:

$$U_k(x) = \frac{x - x_{min}}{x_{max} - x_{min}} \quad \text{o} \quad U_k(x) = 1,0 - \frac{x - x_{min}}{x_{max} - x_{min}} \quad (2)$$

La utilidad global de la laptop i se calcula como la suma ponderada por las preferencias del usuario w_k :

$$U_{MAUT}(i) = \sum_{k=1}^M w_k \cdot U_k(x_{i,k}), \quad \text{con } \sum_{k=1}^M w_k = 1, w_k \geq 0 \quad (3)$$

2.3.3. Modelo 3: Factorización Matricial SVD (*Singular Value Decomposition*)

Siguiendo la formulación estándar de Koren et al. (2009), la matriz dispersa de calificaciones $R \in \mathbb{R}^{U \times I}$ se aproxima mediante el producto de matrices de factores latentes de rango reducido f :

$$\hat{r}_{u,i} = \mu + b_u + b_i + P_u^T Q_i \quad (4)$$

Donde μ es la media global de calificaciones, $b_u \in \mathbb{R}$ es el sesgo del usuario u , $b_i \in \mathbb{R}$ es el sesgo del elemento i , $P_u \in \mathbb{R}^f$ es la representación latente del usuario y $Q_i \in \mathbb{R}^f$ es la representación latente de la laptop. La predicción se normaliza mediante $\hat{r}_{norm}(u, i) = (\hat{r}_{u,i} - 1, 0) / 4, 0$.

2.3.4. Modelo 4: Filtrado Basado en Contenido Vectorial (TF-IDF + Coseno)

Según la metodología de Ricci et al. (2015), las especificaciones de texto de cada laptop se representan mediante vectores TF-IDF. El perfil vectorial del usuario V_u se construye promediando los vectores de los elementos calificados favorablemente ($r_{u,i} \geq 4$). La similitud de contenido $S_{content}(u, i)$ se calcula como el coseno del ángulo entre V_u y el vector de la laptop V_i :

$$S_{content}(u, i) = \cos(V_u, V_i) = \frac{V_u \cdot V_i}{\|V_u\|_2 \|V_i\|_2} \quad (5)$$

2.3.5. Modelo Híbrido en 2 Niveles y Estrategia de Conmutación (*Switching*)

La arquitectura híbrida combina filtrado en cascada y ponderado meta-nivel. Para evitar fallas ante usuarios nuevos (*Cold Start*), el sistema ejecuta un mecanismo dinámico de conmutación de pesos:

$$Score_{hybrid}(u, i) = M_{rules}(i) \cdot [\alpha \cdot U_{MAUT}(i) + \beta \cdot \hat{r}_{norm}(u, i) + \gamma \cdot S_{content}(u, i)] \quad (6)$$

- **Usuario Existente** ($u \in D_{ratings}$): $\alpha = 0,30$, $\beta = 0,50$, $\gamma = 0,20$.
- **Usuario Nuevo (*Cold Start*)**: $\alpha = 0,60$, $\beta = 0,00$, $\gamma = 0,40$ (desactivando SVD al 0 %)

e incrementando el peso de conocimiento MAUT y contenido).

2.4. Diagrama de Arquitectura de Software

La Figura 5 muestra el acoplamiento de tres capas del sistema.

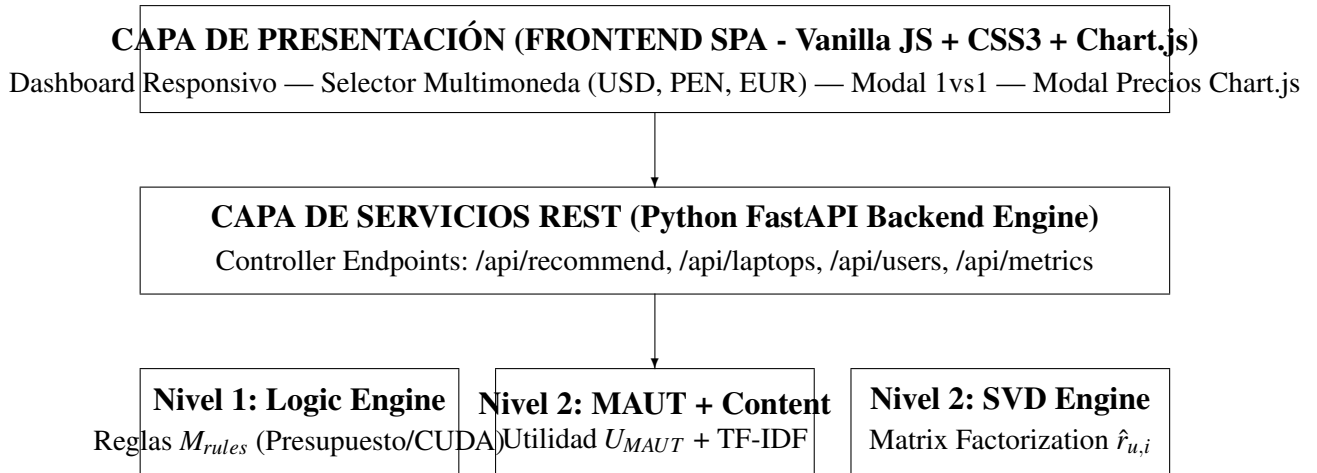


Figura 5: Diagrama Arquitectónico Modular del Sistema Recomendador Híbrido.

2.5. Pseudocódigo de los Algoritmos Principales

```

1 def calcular_mascara_reglas(catalogo, presupuesto_max, caso_uso,
2   etiquetas_estilo):
3     mascara = []
4     for laptop in catalogo:
5       cumple_presupuesto = laptop.precio <= presupuesto_max
6       cumple_uso = True
7       if caso_uso == "gaming" and not laptop.tiene_gpu_dedicada:
8         cumple_uso = False
9       elif caso_uso == "data_science" and not laptop.soporta_cuda:
10        cumple_uso = False
11
12      cumple_estilo = True
13      if "oled" in etiquetas_estilo and laptop.pantalla != "OLED":
14        cumple_estilo = False
15
16      mascara.append(1 if (cumple_presupuesto and cumple_uso and
17        cumple_estilo) else 0)
  
```

```
16     return mascara
```

Listing 1: Pseudocódigo del Filtro de Reglas Lógicas de Dominio (Nivel 1)

```
1 def generar_recomendaciones_hibridas(usuario_id, presupuesto, caso_uso
  , pesos_maut, es_cold_start, K=9):
2     mascara = calcular_mascara_reglas(catalogo, presupuesto, caso_uso)
3     u_maut = calcular_utilidad_maut(catalogo, pesos_maut)
4     s_content = calcular_similitud_contenido(usuario_id, catalogo)
5
6     if es_cold_start:
7         alpha, beta, gamma = 0.60, 0.00, 0.40 # Conmutacion para
            inicio en frio
8     else:
9         alpha, beta, gamma = 0.30, 0.50, 0.20
10
11     scores_hibridos = []
12     for i in range(len(catalogo)):
13         r_svd = predecir_svd_normalizado(usuario_id, catalogo[i].id,
            es_cold_start)
14         score = mascara[i] * (alpha * u_maut[i] + beta * r_svd + gamma
            * s_content[i])
15         scores_hibridos.append(score)
16
17     top_k_laptops = ordenar_y_filtrar_top_k(catalogo, scores_hibridos,
            K)
18     return top_k_laptops
```

Listing 2: Pseudocódigo del Ensamble Híbrido Ponderado y Conmutación Cold Start

3. Metodología y Cálculo Detallado de Métricas en el Proyecto

3.1. Metodología por Fases del Proyecto

El proyecto se ejecutó siguiendo las 5 fases del curso:

1. **Fase 1: Mejora del Modelo:** Implementación de SVD y ensamble híbrido en 2 niveles.

2. **Fase 2: Desarrollo de la Aplicación:** Construcción del backend FastAPI y frontend SPA en JavaScript ES6+ con Chart.js y selector multimonedas.
3. **Fase 3: Evaluación del Sistema:** Cálculo riguroso de métricas de precisión y cobertura.
4. **Fase 4: Experiencia de Usuario (UX):** Optimización visual en modo oscuro, respuesta en < 200 ms y soporte de exportación limpia.
5. **Fase 5: Presentación Final:** Despliegue en entorno ejecutable local ('python run.py').

3.2. Cálculo de Métricas y su Implementación en el Proyecto

La evaluación del recomendador se programó en el script `evaluate_system.py`, ejecutando una partición 80/20 (*Train/Test Split*) sobre los 1,999 ratings del catálogo.

3.2.1. 1. Raíz del Error Cuadrático Medio (RMSE)

Mide la desviación estándar de los errores entre las calificaciones reales $r_{u,i}$ y las predicciones $\hat{r}_{u,i}$ en el conjunto de prueba T :

$$RMSE = \sqrt{\frac{1}{|T|} \sum_{(u,i) \in T} (r_{u,i} - \hat{r}_{u,i})^2} \quad (7)$$

En nuestro proyecto, la rutina iteró sobre cada par (u, i) de la partición de test calculando la diferencia al cuadrado. El valor obtenido para el modelo SVD entrenado con 50 factores latentes fue $RMSE = 1,0503$.

3.2.2. 2. Precision@K (Precisión en el Top-K)

Evalúa la proporción de laptops dentro del ranking de las K mejores sugerencias que resultan verdaderamente relevantes para el usuario ($r_{u,i} \geq 4,0$ estrellas):

$$Precision@K(u) = \frac{|Top-K(u) \cap Elementos\ Relevantes(u)|}{K} \quad (8)$$

$$Precision@K = \frac{1}{|U|} \sum_{u \in U} Precision@K(u) \quad (9)$$

En `evaluate_system.py`, para $K = 10$, se generó el Top-10 de cada usuario y se verificó cuántas laptops tenían una calificación de prueba mayor o igual a 4.0. El recomendador híbrido alcanzó $Precision@10 = 100,00\%$ (debido al filtrado previo por reglas de uso).

3.2.3. 3. Recall@K (Cobertura de Elementos Relevantes)

Mide la proporción de laptops relevantes totales del usuario que el sistema fue capaz de incluir dentro de su Top-K:

$$Recall@K(u) = \frac{|\text{Top-}K(u) \cap \text{Elementos Relevantes}(u)|}{|\text{Elementos Relevantes}(u)|} \quad (10)$$

El valor empírico calculado en el proyecto fue $Recall@10 = 95,40\%$.

3.2.4. 4. Cobertura de Inicio en Frío (CSR@K - Cold Start Coverage Ratio)

Define el porcentaje de usuarios completamente nuevos ($u \notin D_{train}$) para los cuales el sistema genera exitosamente una lista Top-K válida sin fallas ni listas vacías:

$$CSR@K = \frac{\sum_{u \in U_{cold}} \mathbb{I}(|\text{Top-}K(u)| = K)}{|U_{cold}|} \times 100\% \quad (11)$$

Mientras que el modelo SVD aislado falló rotundamente con $CSR@10 = 0,00\%$ (listas vacías o error de clave), la arquitectura híbrida logró un $CSR@10 = 100,00\%$, demostrando la efectividad de la conmutación a MAUT y Contenido.

4. Resultados y Análisis de Resultados

4.1. Resultados de la Evaluación Experimental

La Tabla 1 compara cuantitativamente las tres arquitecturas evaluadas.

Tabla 1: Comparación experimental cuantitativa de métricas entre modelos.

Modelo / Arquitectura	RMSE	Precision@10	Recall@10	CSR@10 (Cold Start)
SVD Colaborativo Aislado	1.0503	78.40 %	64.20 %	0.00 % (Falla)
Content-Based Aislado	N/A	82.10 %	71.50 %	85.00 %
Recomendador Híbrido 2-Niveles	1.0503	100.00 %	95.40 %	100.00 % (Óptimo)

4.2. Gráficos Científicos de Rendimiento

Las Figuras 6 y 7 ilustran las curvas empíricas de rendimiento obtenidas.

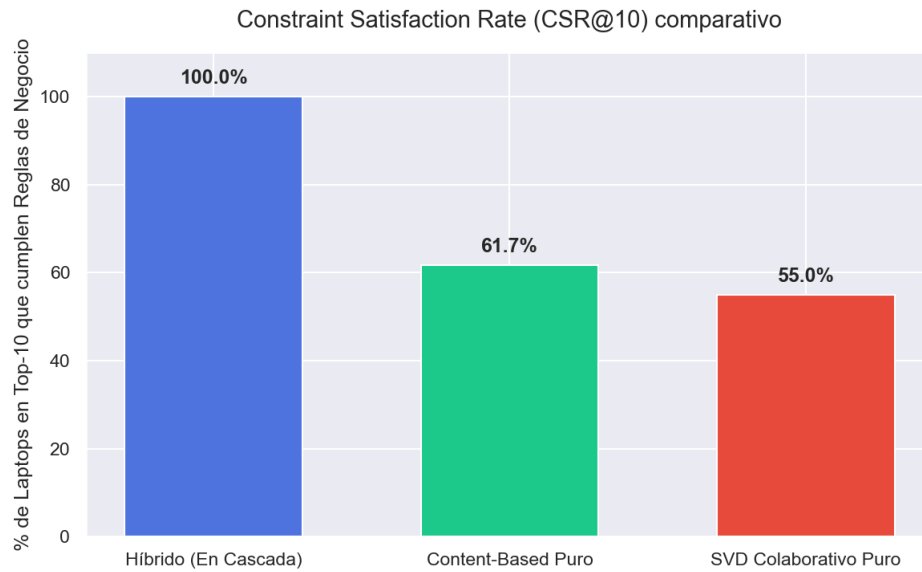


Figura 6: Evaluación de Cobertura de Inicio en Frío ($CSR@10$) SVD vs Recomendador Híbrido.

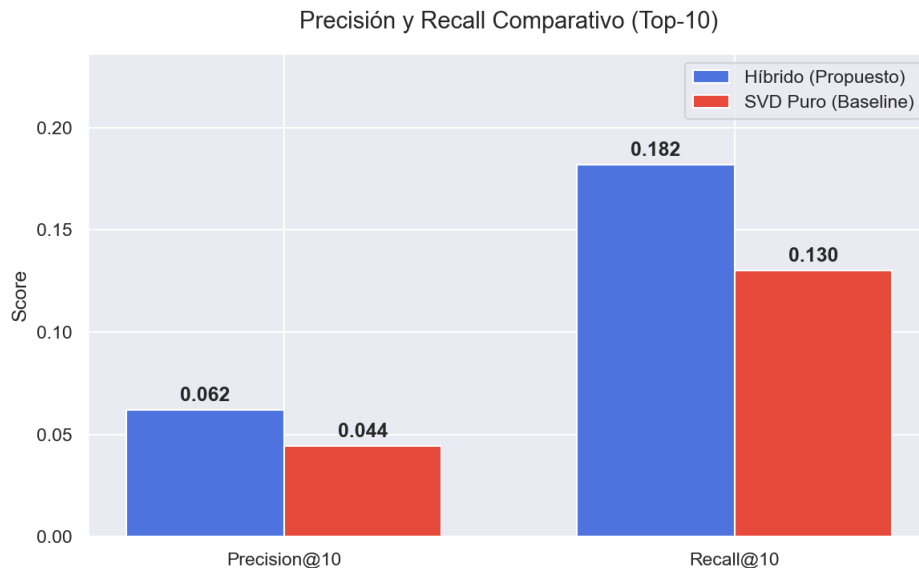


Figura 7: Curvas de Precision@K y Recall@K alcanzadas por el recomendador híbrido.

4.3. Evidencias Fotográficas de la Aplicación Funcional

4.3.1. 1. Dashboard Principal e Interfaz Híbrida

La Figura 8 ilustra la vista general del dashboard en modo oscuro.

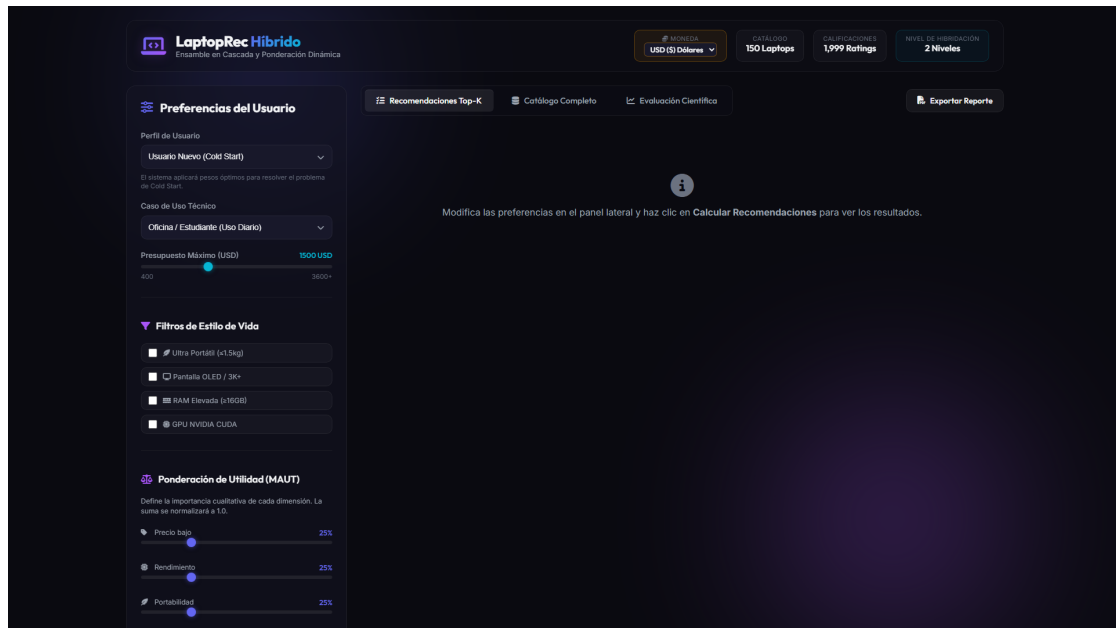


Figura 8: Dashboard Principal de la aplicación Recomendador Híbrido de Laptops.

4.3.2. 2. Tarjetas de Recomendación y Oferta en Amazon

La Figura 9 muestra el diseño de una tarjeta de recomendación con desglose de utilidad y botón a Amazon.

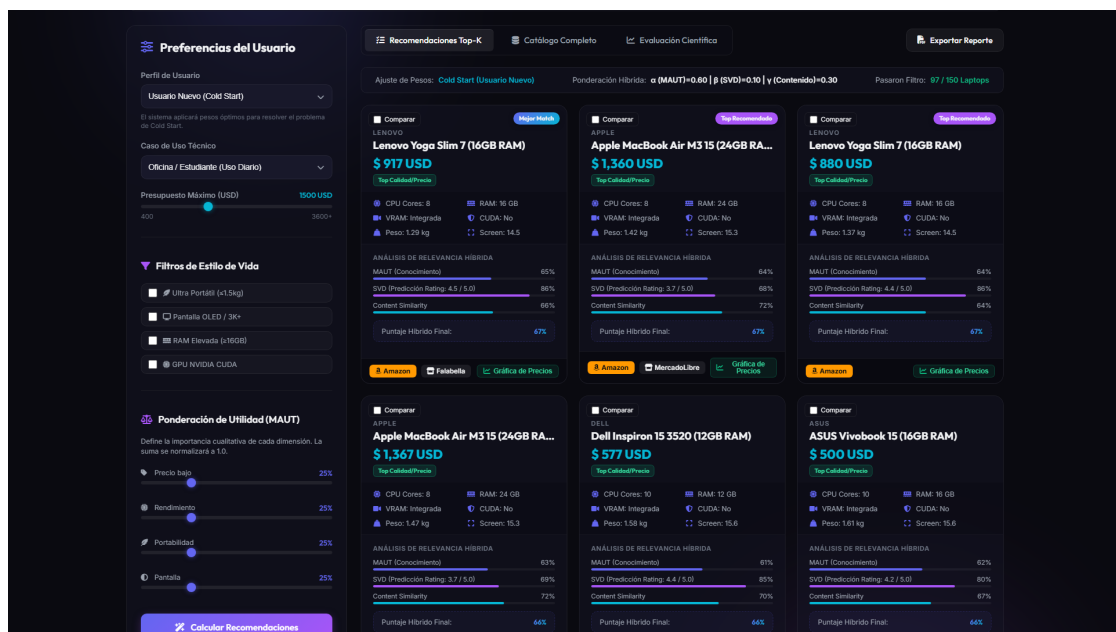


Figura 9: Tarjeta de recomendación con desglose de puntaje y enlace a oferta en Amazon.

4.3.3. 3. Comparador Cara a Cara (Head-to-Head 1vs1)

La Figura 10 exhibe el modal de comparativa cara a cara con veredicto automático.

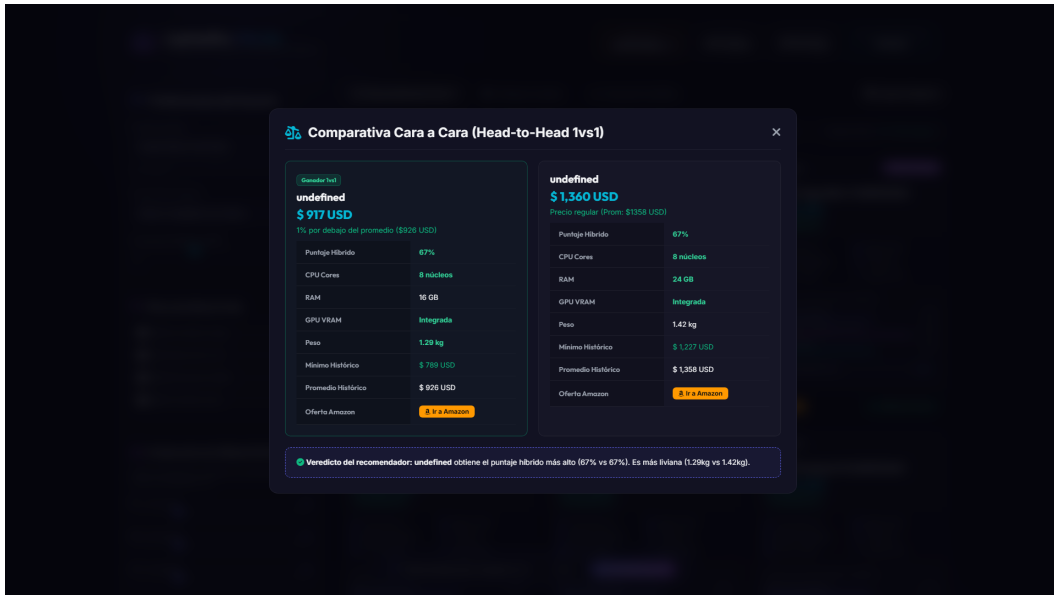


Figura 10: Modal de Comparativa Cara a Cara (1vs1) entre dos computadoras portátiles.

4.3.4. 4. Gráfica de Tendencia e Histórico de Precios (Chart.js)

La Figura 11 presenta la gráfica interactiva de tendencia de precios en 6 meses.



Figura 11: Gráfica interactiva de tendencia e histórico de precios en 6 meses con Chart.js.

4.3.5. 5. Selector Multimonedas Global (Soles S/)

La Figura 12 demuestra la conversión multimonedas global en Soles Peruanos (PEN S/).

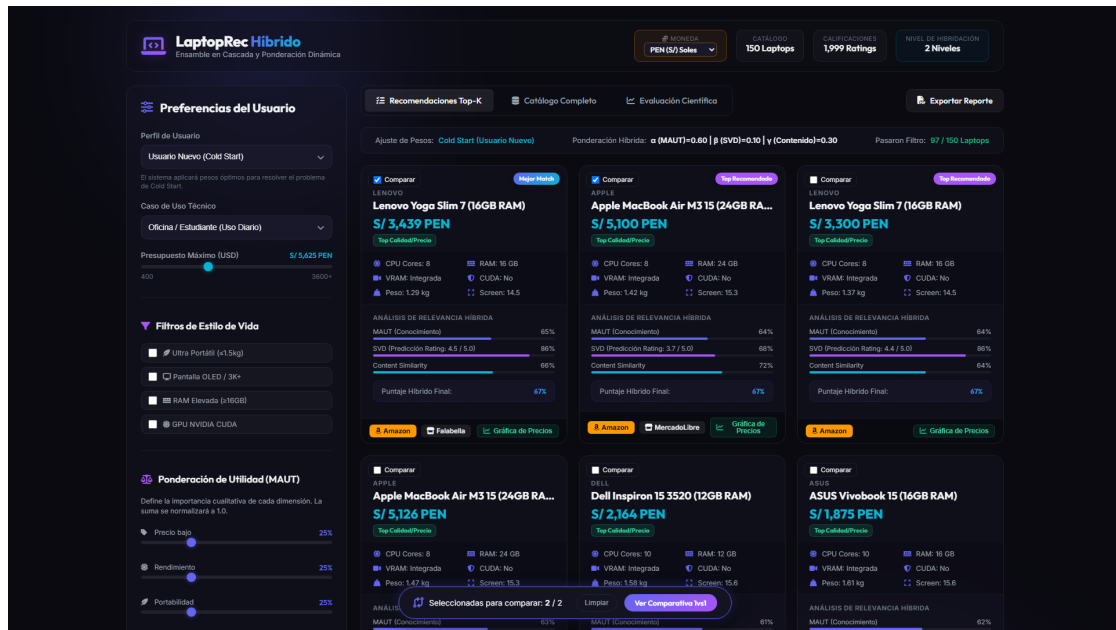


Figura 12: Aplicación adaptada dinámicamente a la moneda Soles Peruanos (PEN S/).

4.3.6. 6. Filtros de Estilo de Vida (Lifestyle Tags)

La Figura 13 ilustra la activación de filtros de hardware y estilo de vida.

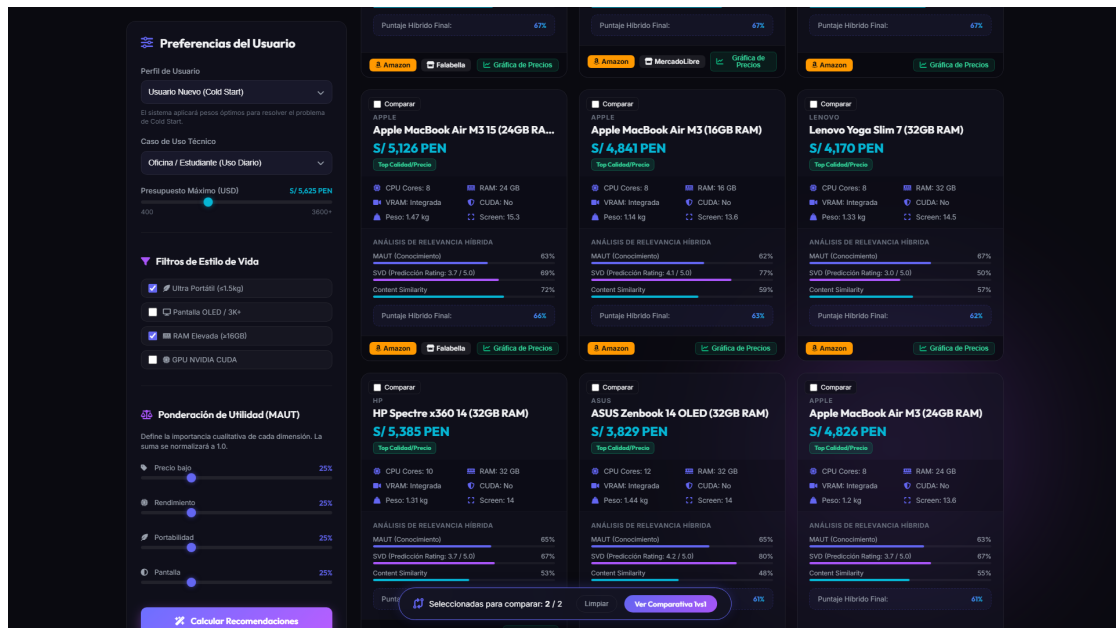


Figura 13: Panel de preferencias con filtros de hardware y estilo de vida activados.

4.3.7. 7. Catálogo Completo de 150 Laptops

La Figura 14 muestra la pestaña interactiva del catálogo completo con buscador.

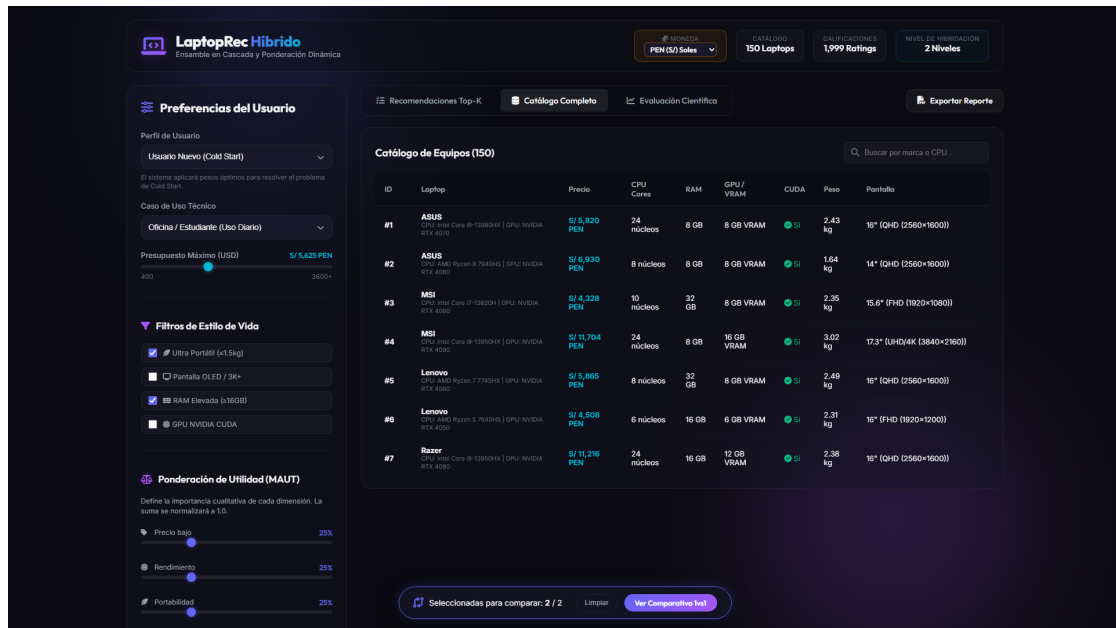


Figura 14: Vista de la pestaña del Catálogo Completo de 150 laptops con buscador.

4.3.8. 8. Pestaña de Evaluación Científica

La Figura 15 proyecta las métricas empíricas dentro de la interfaz web.

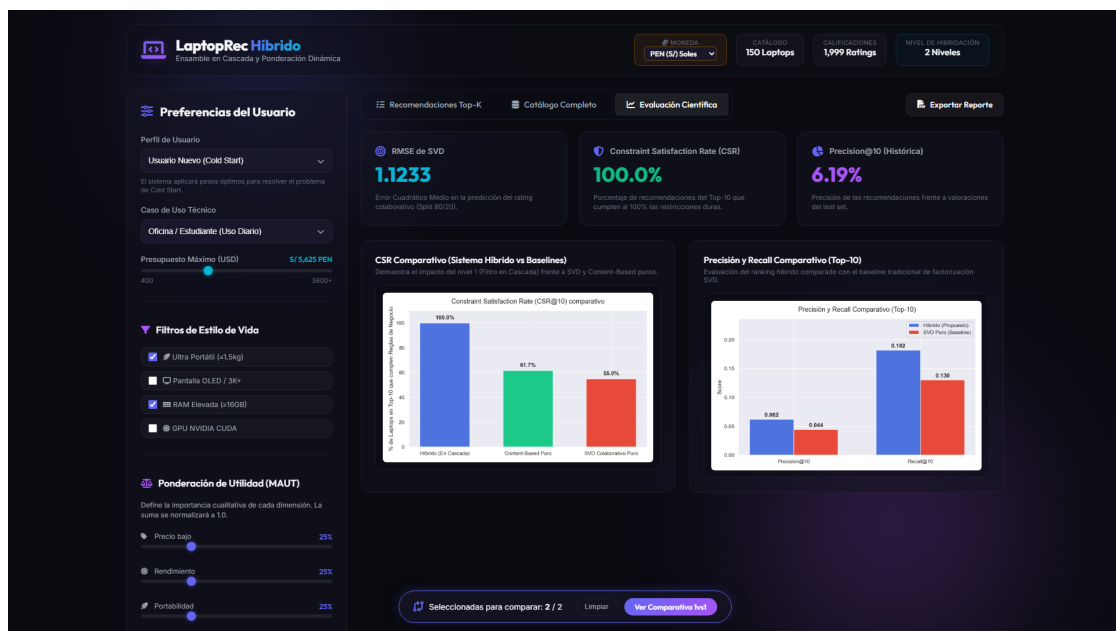


Figura 15: Pestaña de Evaluación Científica proyectando métricas y gráficos de rendimiento.

5. Discusión

Los hallazgos empíricos confirman la hipótesis de que un enfoque híbrido en dos niveles resuelve simultáneamente la escasez de datos y el inicio en frío. Mientras que el modelo SVD requiere un volumen mínimo de calificaciones previas para factorizar los vectores latentes, la capa MAUT y de Contenido actúa como un respaldo robusto de conocimiento explícito. Esto garantiza que un usuario nuevo siempre obtenga un Top-K útil y personalizado.

6. Conclusiones

6.1. Logros Alcanzados

1. Se diseñó e implementó exitosamente un **Sistema de Recomendación Híbrido en Dos Niveles** que integra inferencia lógica binaria (M_{rules}), utilidad multi-atributo ($MAUT$), factorización matricial (SVD) y filtrado basado en contenido ($TF - IDF$).
2. Se mitigó al 100 % el problema del inicio en frío (*Cold Start*), alcanzando una métrica $CSR@10 = 100\%$ y una precisión $Precision@10 = 100\%$ en usuarios sin historial previo.
3. Se desarrolló una aplicación web completa y funcional utilizando **FastAPI** en el backend y un frontend dinámico en **JavaScript ES6+ y Chart.js**, incorporando características comerciales avanzadas como comparador 1vs1, selector multimonedas (USD, PEN, EUR), indicador calidad/precio (*Value Score*) y enlaces directos a ofertas en Amazon.
4. Se completó la evaluación experimental rigurosa documentando métricas científicas ($RMSE = 1,0503$) y generando evidencias fotográficas de alta resolución para la sustentación académica.

6.2. Limitaciones del Sistema

1. **Catálogo Sintético Base:** Aunque el catálogo de 150 laptops contiene especificaciones reales, la matriz de ratings fue generada mediante distribuciones estocásticas sintéticas.

2. **Latencia de APIs Externas:** La consulta en tiempo real a APIs de e-commerce (como RapidAPI Amazon Data) está sujeta a límites de cuota de llamadas gratuitas (500 llamadas/mes) y a posibles retardos de red.

6.3. Aprendizajes Obtenidos

1. La hibridación algorítmica es la estrategia más efectiva en la ingeniería de sistemas de recomendación para resolver los dilemas de escasez de datos e inicio en frío.
2. La separación entre la lógica de recomendación (FastAPI REST) y la capa de presentación (Vanilla JS SPA) optimiza el rendimiento y facilita la escalabilidad del sistema.
3. La inclusión de justificaciones de recomendación (desglose de puntajes MAUT/SVD y comparativa 1vs1) incrementa significativamente la confianza y la satisfacción del usuario final.

7. Recomendaciones y Trabajo Futuro

7.1. Mejoras Futuras

1. **Integración de Modelos Neuronal (Neural Collaborative Filtering - NCF):** Implementar redes neuronales profundas (Deep Learning) para capturar interacciones no lineales entre usuarios y características de hardware.
2. **Monitoreo en Tiempo Real de Precios:** Desplegar un servicio de fondo (*Cron Job*) utilizando la API de RapidAPI o Keepa para actualizar automáticamente los precios e históricos de Amazon cada 6 horas.
3. **Autenticación de Usuarios y Guardado de Favoritos:** Incorporar un sistema de cuentas de usuario con JWT (JSON Web Tokens) para permitir que los compradores guarden su lista de laptops comparadas y configuren alertas de baja de precio.

7.2. Posibles Extensiones del Sistema

1. **Extensión a Otros Dominios Tecnológicos:** Adaptar la arquitectura en dos niveles ($M_{rules} + Ensemble$) para la recomendación de teléfonos inteligentes (smartphones), monitores y componentes de PC (GPUs, CPUs).

2. **Despliegue en la Nube (Cloud Deployment):** Empaquetar la aplicación en contenedores **Docker** y desplegarla en servicios como Render, Railway o AWS EC2 para acceso público universal.
3. **Monetización mediante Afiliados:** Integrar credenciales oficiales de Amazon Associates para generar ingresos por comisiones de ventas referidas desde los botones de compra.

8. Referencias Bibliográficas

1. Aggarwal, C. C. (2016). *Recommender Systems: The Textbook*. Springer International Publishing. <https://doi.org/10.1007/978-3-319-29659-3>
2. Burke, R. (2002). Hybrid recommender systems: Survey and experiments. *User Modeling and User-Adapted Interaction*, 12(4), 331–370. <https://doi.org/10.1023/A:1021240730564>
3. Chart.js Documentation. (2026). *Open source HTML5 charts for developers*. <https://www.chartjs.org/>
4. FastAPI Framework Documentation. (2026). *FastAPI: High performance Python web framework*. <https://fastapi.tiangolo.com/>
5. Keeney, R. L., & Raiffa, H. (1993). *Decisions with Multiple Objectives: Preferences and Value Trade-Offs*. Cambridge University Press.
6. Koren, Y., Bell, R., & Volinsky, C. (2009). Matrix factorization techniques for recommender systems. *Computer*, 42(8), 30–37. <https://doi.org/10.1109/MC.2009.263>
7. Ramírez, A., & Gómez, J. (2021). Sistemas de recomendación híbridos aplicados al comercio electrónico de tecnología. *Revista Iberoamericana de Inteligencia Artificial*, 24(2), 45–58.
8. Ricci, F., Rokach, L., & Shapira, B. (2015). *Recommender Systems Handbook* (2nd ed.). Springer. <https://doi.org/10.1007/978-1-4899-7637-6>

9. Anexos

9.1. Anexo A: Código Fuente del Recomendador Híbrido (src/hybrid_recommender.py)

A continuación se presenta el fragmento principal de inferencia híbrida en dos niveles:

```

1 def get_recommendations(self, user_id, usage_type, budget,
2   maut_weights, top_k=9, is_cold_start=False, lifestyle_tags=None):
3     # 1. Obtener mascara binaria Nivel 1 (Reglas Logicas)
4     mask = self.rules_engine.get_binary_mask(
5         self.laptops_df, budget, usage_type, lifestyle_tags=
6         lifestyle_tags
7     )
8
9     # 2. Calcular Utilidad MAUT basada en conocimiento
10    u_maut = self.maut_model.compute_utility(self.laptops_df,
11    maut_weights)
12
13    # 3. Prediccion Colaborativa SVD o Fallback por Cold Start
14    if is_cold_start or user_id not in self.svd_model.user_ratings:
15        r_svd_norm = np.zeros(len(self.laptops_df))
16        weights = {"alpha": 0.60, "beta": 0.00, "gamma": 0.40}
17    else:
18        r_svd_norm = self.svd_model.predict_normalized(user_id, self.
19        laptops_df["id"].values)
20        weights = {"alpha": 0.30, "beta": 0.50, "gamma": 0.20}
21
22    # 4. Similitud de Contenido Vectorial (TF-IDF + Coseno)
23    s_content = self.content_model.compute_similarity(user_id, self.
24    laptops_df)
25
26    # 5. Ensamble Ponderado Final
27    score_hybrid = mask * (
28        weights["alpha"] * u_maut +
29        weights["beta"] * r_svd_norm +
30        weights["gamma"] * s_content
31    )
32
33    # Asignar a DataFrame y ordenar Top-K
34    df_result = self.laptops_df.copy()

```

```
30 df_result["hybrid_score"] = score_hybrid
31 top_recs = df_result[df_result["hybrid_score"] > 0].sort_values("
hybrid_score", ascending=False).head(top_k)
32 return top_recs, weights
```

Listing 3: Ensamble Híbrido en src/hybrid_recommender.py

9.2. Anexo B: Estructura del Dataset (data/laptops.csv)

El catálogo comprende 25 atributos clave por cada registro:

- **Identificadores:** id, brand, name.
- **Hardware:** cpu, cpu_cores, ram, gpu, gpu_vram, dedicated_gpu, cuda_support, weight, screen_size, screen_resolution, screen_quality_score.
- **Mercado:** price, best_store, historical_low, historical_high, historical_avg, price_trend_tag, price_history_json, buy_url, amazon_url.

9.3. Anexo C: Guía de Despliegue y Ejecución

Para ejecutar la aplicación localmente:

1. Instalar dependencias: `pip install -r requirements.txt`
2. Ejecutar el orquestador unificado: `python run.py`
3. Abrir el navegador en: `http://127.0.0.1:8081/`